

glTF Format for G-PCC Encoded Gaussian Splats

Isaac Yang*
Sungkyunkwan University

Yeong-Gyu Kim†
Sungkyunkwan University

Seung Huk Ahn‡
Sungkyunkwan University

Eun-Seok Ryu§
Sungkyunkwan University

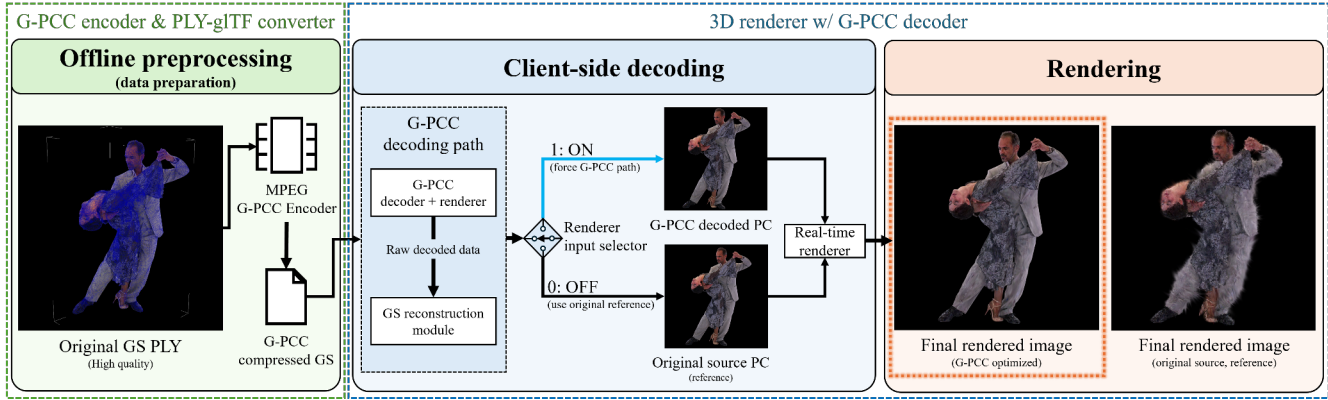


Figure 1: Conceptual diagram of glTF format for G-PCC encoded Gaussian Splats

ABSTRACT

As the demand for high-fidelity six degrees of freedom (6DoF) content grows, 3D Gaussian Splatting (3DGS) has emerged as a promising representation due to its superior rendering quality and speed. However, the massive storage requirements of 3DGS pose significant challenges for efficient transmission and web-based streaming. While the MPEG geometry-based point cloud compression (G-PCC) standard offers efficient compression for point clouds, its direct application to 3DGS is hindered by data type mismatches and the lack of a standardized container. This paper proposes a standard-compliant streaming framework that encapsulates G-PCC encoded 3DGS bitstreams within the glTF format. This paper introduces a quantization-aware encoding scheme and a custom glTF extension MPEG_GSC_EXT to preserve the float-precision attributes of Gaussians. Furthermore, we implement a client-side rendering pipeline capable of on-the-fly de-quantization. Experimental results show the proposed method achieves significant file size reduction compared to raw PLY and compressed formats while maintaining visual fidelity.

Index Terms: 3D Gaussian Splatting (3DGS), geometry-based point cloud compression (G-PCC), GL transmission format (glTF).

1 INTRODUCTION

Recently, virtual reality (VR) and extended reality (XR) technologies have advanced rapidly, increasing the demand for photorealistic 6DoF immersive media[7]. To support these experiences, MPEG has developed standards such as MPEG Immersive Video (MIV) and Video-based Dynamic Mesh Coding (VDMC)[2]. While these standards handle mesh and multi-view video efficiently, 3D Gaussian Splatting (3DGS) has gained attention for

its ability to represent complex scenes without explicit geometry, using millions of 3D Gaussians[6].

However, the explicit nature of 3DGS results in prohibitive file sizes, often exceeding hundreds of megabytes per scene. This creates a bottleneck for real-time streaming over bandwidth-constrained networks[5]. Although G-PCC (ISO/IEC 23090-9) is the industry standard for compressing static point clouds, it operates primarily on integer attributes[3]. This creates a discrepancy when compressing 3DGS data, which relies on high-precision floating-point attributes (e.g., covariance, opacity) for accurate splatting. Moreover, there is no standardized method to embed G-PCC bitstreams into glTF, the de facto standard for 3D web delivery[9].

To address these issues, this paper proposes a unified pipeline for compressing and streaming 3DGS content using G-PCC and glTF. This paper primarily introduces an attribute quantization scheme designed to adapt floating-point Gaussian parameters to the G-PCC integer-based pipeline. Building on this, this paper defines a custom glTF extension that encapsulates the G-PCC bitstreams and necessary quantization metadata within a standard container. Furthermore, this paper presents a modified rendering framework capable of performing on-the-fly de-quantization and attribute restoration to ensure real-time playback.

2 RELATED WORK

2.1 3D Gaussian Splatting

3D Gaussian Splatting (3DGS) has established itself as a premier representation for real-time radiance field rendering, offering quality comparable to neural radiance fields (NeRF) without the high inference cost[8]. However, the explicit nature of 3DGS requires storing millions of primitives with high-dimensional attributes (position, covariance, opacity, spherical harmonics), often resulting in scene sizes exceeding gigabytes. This storage overhead creates a critical bottleneck for streaming applications, necessitating efficient compression strategies similar to those used in immersive video.

2.2 MPEG point cloud compression standardization

The MPEG G-PCC standard (ISO/IEC 23090-9) provides a robust framework for compressing sparse geometry and attributes. Although G-PCC is highly effective for LiDAR- or photogrammetry-based point clouds using octrees and lifting

*e-mail: icarax1491@skku.edu

†e-mail: kyk2798@skku.edu

‡e-mail: powerminers@skku.edu

§e-mail: esryu@skku.edu

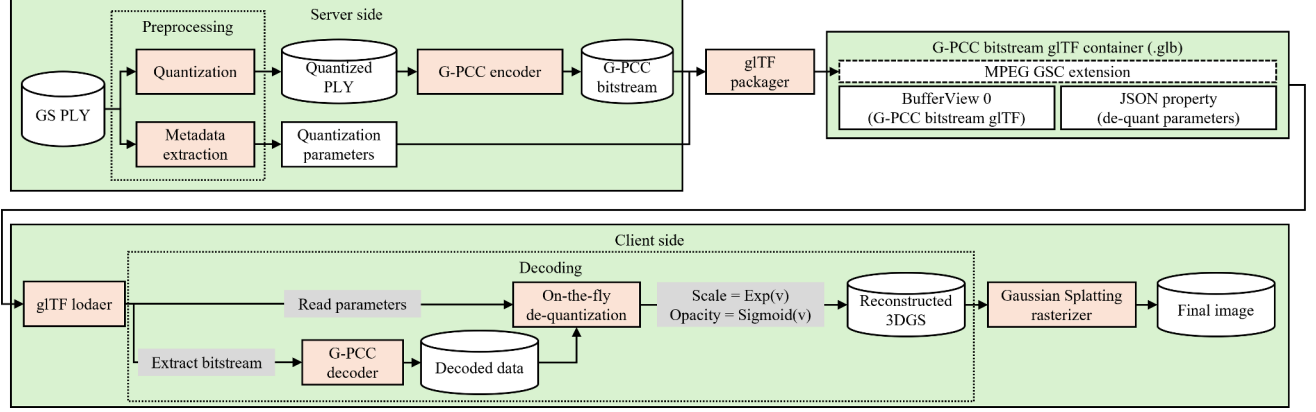


Figure 2: System diagram of G-PCC encoding and GS PLY bitstream glTF rendering

schemes, it is fundamentally designed for integer signals. This creates a significant domain mismatch for 3DGS, which relies on sensitive floating-point values to define anisotropic ellipsoids. Recent studies have attempted to bridge this gap via quantization, but achieving high compression ratios without introducing visual artifacts—such as “popping” or incorrect blending—remains a challenge[4].

2.3 Gaussian Splats glTF

The glTF 2.0 standard is the foundation of 3D content delivery on the web. Although mesh compression extensions such as Draco exist, they are not optimized for the splat-based rendering pipeline. Recent proposals within MPEG have suggested mapping Gaussian attributes directly to glTF accessors to ensure compatibility[1]. However, these methods typically handle uncompressed or lightly compressed data. There is currently no standardized container format that encapsulates the raw G-PCC bitstream within glTF to leverage MPEG’s compression efficiency while maintaining the portability of the web 3D ecosystem.

3 PROPOSED METHOD

This section presents a framework to bridge the domain discrepancy between floating-point 3DGS attributes and the integer-based

MPEG G-PCC standard. Fig. 2 illustrates the end-to-end system pipeline.

3.1 Quantization and G-PCC Encoding Strategy

The core challenge lies in mapping high-precision 3DGS attributes (position μ , covariance Σ , opacity α , SH C) to the discrete integer space required by G-PCC (ISO/IEC 23090-9). Proposed method employs a transformation pipeline $\mathcal{T}$ consisting of normalization and uniform quantization. For an attribute channel x with global extrema x_{min} and x_{max} , the quantized integer $\hat{x}_{int}$ for a target bit-depth B is derived as:

$$\hat{x}_{int} = \text{floor} \left(\frac{x - x_{min}}{x_{max} - x_{min}} \cdot (2^B - 1) + 0.5 \right)$$

Proposed method adopts an adaptive bit-depth allocation strategy: higher precision ($B = 12$) is assigned to geometry and DC components to preserve structure, while lower precision ($B = 10 \sim 11$) is used for high-frequency SH and opacity to maximize compression. For geometry coding, Gaussian means μ are mapped to the G-PCC octree coordinate system using a quantization scale Δ_s , effectively sorting data by spatial locality:

$$\mu_{int} = \text{round} \left(\frac{\mu - \mu_{offset}}{\Delta_s} \right)$$

This spatial sorting allows the G-PCC attribute coder (using RAHT or Lifting) to effectively decorrelate the quantized signal $\hat{x}_{int}$, yielding a compact bitstream.

3.2 glTF Extension Schema: MPEG_GSC_EXT

To encapsulate the encoded stream without transcoding overhead, proposed method defines a custom glTF extension, MPEG_GSC_EXT. Fig. 3 shows the example of glTF format and MPEG_GSC_EXT extension. Attached to the mesh.primitive object, it treats the bitstream as an opaque payload using two components:

- **bitstreamIndex**: Reference to a bufferView containing the raw G-PCC binary for efficient fetching.
- **quantizationInfo**: Metadata dictionary ($\{x_{min}, x_{max}, B\}$) for each attribute, serving as the key for physical reconstruction.

```

1 {
2   "accessors": [1],
3   "asset": {
4     "generator": "ply2gltf converter",
5     "version": "2.0"
6   },
7   "bufferViews": [
8     {
9       "buffer": 0,
10      "byteLength": 2375959,
11      "byteOffset": 0,
12      "extensions": {
13        "EXT_gpsc_compression": {
14          "boundingBox": {
15            "max": [
16              254.66094970793125,
17              253.73081970214844,
18              251.25576782226562
19            ],
20            "min": [
21              -1.3390474310458008,
22              -2.269181251525879,
23              -4.744227886199951
24            ]
25          },
26          "buffer": 0,
27          "byteLength": 2375959,
28          "byteOffset": 0,
29          "decodingParams": {
30            "attributeCodec": "RAHT",
31            "geometryCodec": "TMC13"
32          },
33          "profile": "G-PCC",
34          "profileLevel": "TMC13v22",
35          "splatCount": 0
36        }
37      },
38      "extensionsUsed": [
39        "EXT_gaussian_splats",
40        "EXT_gpsc_compression"
41      ],
42      "nodes": [
43        {
44          "scene": 0,
45          "scenes": [
46            {
47              "nodes": [
48                0
49              ]
50            }
51          ]
52        }
53      ]
54    }
55  ],
56  "mesh": {
57    "primitives": [
58      {
59        "attributes": {
60          "position": 0,
61          "normal": 0,
62          "texCoord_0": 0,
63          "opacity": 0,
64          "SH": 0
65        },
66        "indices": 0,
67        "material": 0
68      }
69    ]
70  }
71 }
```

Figure 3: glTF format and MPEG_GSC_EXT extension

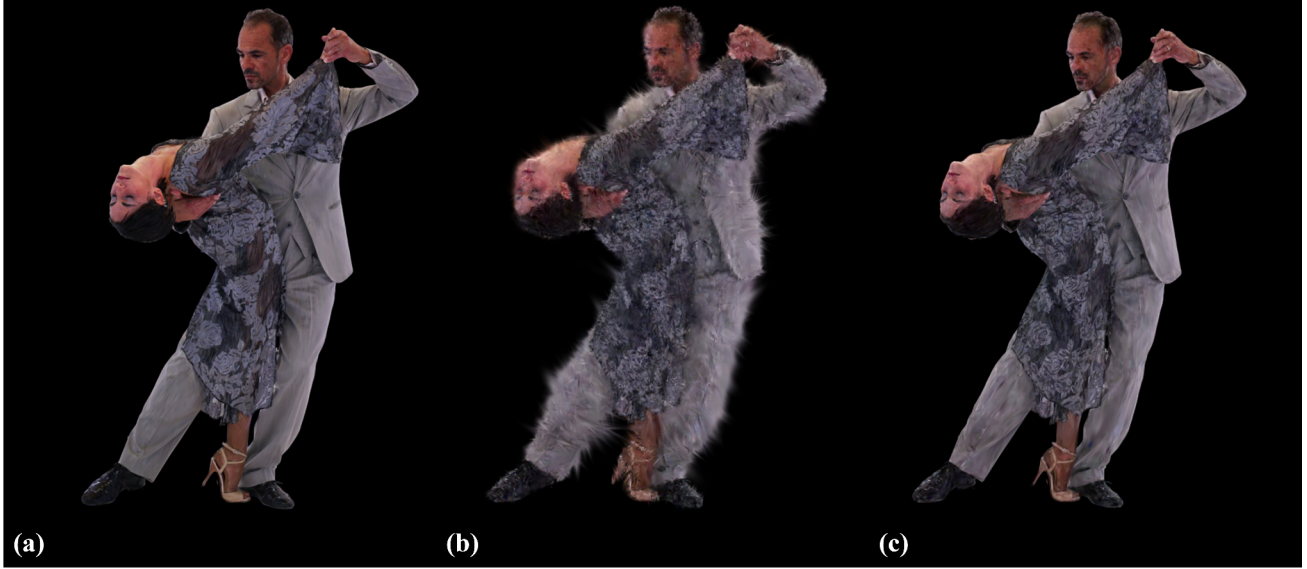


Figure 4: Subjective quality evaluation with *Tango_duo* sequence

(a) Original GS PLY

(b) Rendered G-PCC GS PLY bitstream glTF referring original GS PLY

(c) Rendered G-PCC GS PLY bitstream glTF which forces the renderer to reconstruct the scene exclusively from the bitstream

3.3 Client-side Reconstruction and Activation

The client performs inverse quantization followed by non-linear activation to restore physical properties. Crucially, to ensure mathematical stability, scaling (S) and opacity (α) are quantized in logarithmic and logit domains, respectively.

First, the floating-point value $\tilde{x}$ is recovered from the integer stream:

$$\tilde{x} = \frac{\hat{x}_{int}}{2^B - 1} \cdot (x_{max} - x_{min}) + x_{min}$$

Subsequently, domain-specific activation functions $\phi(\cdot)$ are applied. For scale, proposed method uses the exponential function $S = \exp(\tilde{x})$. For opacity, we use the sigmoid function:

$$\alpha = \phi_{opacity}(\tilde{x}) = \frac{1}{1 + e^{-\tilde{x}}}$$

4 EXPERIMENTAL RESULTS

This section evaluates the performance of the proposed G-PCC-based glTF packaging framework. We analyze the compression efficiency, visual fidelity, and rate-distortion characteristics across a diverse set of standard 3DGS datasets, comparing the proposed method against the uncompressed raw PLY baseline.

4.1 Experimental Setup

To ensure a comprehensive evaluation, proposed framework selected nine representative scenes from standard 3DGS datasets, covering a wide range of complexity and scales: *Tango_duo*, *Tango_solo_female*, *Tango_solo_male*, *Ferrari*, *Flowers*, *Cricket_player*, *Bugatti*, *Library*, and *Tennis_player*. The raw PLY files served as the anchor for all comparisons.

For the proposed method, this paper employed the G-PCC reference software (tmc3 v23.0) to encode the quantized point clouds. This framework configured the quantization bit-depth (B) to range from 10 to 12 bits for geometry and attributes, aiming to identify the optimal balance between storage reduction and rendering quality. The evaluation metrics are defined as follows:

- **Compression ratio (CR):** The ratio of the original PLY file size to the final packaged GLB file size (including the glTF container overhead).
- **Bits per point (BPP):** The average number of bits required to represent a single Gaussian primitive.
- **Visual quality:** Peak signal-to-noise ratio (PSNR) and structural similarity (SSIM) measured on rendered viewpoints at 1920x1080 resolution.

4.2 Compression Efficiency

The method achieved a substantial reduction in storage requirements across all datasets, with an average compression ratio of approximately 6.5:1. Notably, the compression performance varied depending on the scene characteristics. The *Tango_duo* sequence exhibited exceptional compressibility, achieving a ratio of 20.8:1, reducing the file size from 49.4 MB to just 2.4 MB. Large-scale scenes such as *Flowers* and *Library* also showed significant savings; for instance, *Flowers* was compressed from 712.3 MB to 126.7 MB (5.6:1), demonstrating the scalability of our approach for heavy assets.

The average BPP was recorded at approximately 23.5 bits, which is drastically lower than the theoretical 200+ bits required for raw floating-point attributes (Position, Scale, Rotation, Opacity, SH). We also analyzed the packaging overhead introduced by the glTF container structure. Our analysis confirms that the overhead (JSON header, extension metadata, and binary padding) accounts for less than 1.0% of the total file size in most cases. This validates that encapsulating the G-PCC bitstream into a standard-compliant glTF container incurs negligible cost while ensuring broad interoperability.

4.3 Visual Quality and Restoration Accuracy

Fig. 4 presents the qualitative results of the rendered scenes. Despite the aggressive quantization and integer-based compression, the proposed on-the-fly de-quantization pipeline successfully restores the Gaussian attributes with high fidelity.

Table 1: Quantitative metrics of rate-distortion analysis across different QP configurations

Sequence Name	QP Cfg	Eff. QP	BPP (bits)	Raw Size (MB)	GLB Size (MB)	Comp. Ratio	glTF Overhead	Viewpoint PSNR (dB)	Viewpoint SSIM	OM PSNR	OM SSIM
<i>Bugatti</i>	4	0	462.9	49.5	11.0	4.5	0.80	46.11	0.9974	45.12	0.9968
	8	4	416.1	49.5	9.9	5.0	0.82	45.99	0.9974	45.00	0.9967
	12	8	374.9	49.5	8.9	5.6	0.83	45.92	0.9973	44.93	0.9966
<i>Cricket_Player</i>	4	0	469.2	48.7	11.0	4.4	0.80	54.34	0.9995	44.77	0.9956
	8	4	422.0	48.7	9.9	4.9	0.82	54.29	0.9995	44.72	0.9956
	12	8	380.4	48.7	8.9	5.5	0.83	54.24	0.9995	44.68	0.9955
<i>Ferrari</i>	4	0	482.5	49.3	11.4	4.3	0.80	44.15	0.9972	44.30	0.9977
	8	4	435.5	49.3	10.3	4.8	0.81	43.95	0.9971	44.08	0.9976
	12	8	393.7	49.3	9.3	5.3	0.83	43.87	0.9971	44.00	0.9976
<i>Flowers</i>	4	0	352.8	712.3	120.8	5.9	0.84	292.11	0.9994	32.22	0.9988
	8	4	301.1	712.3	103.1	6.9	0.86	291.06	0.9993	31.20	0.9987
	12	8	267.6	712.3	91.6	7.8	0.88	290.96	0.9993	30.95	0.9986
<i>Library</i>	4	0	353.2	569.6	101.6	5.6	0.84	48.69	0.9973	48.30	0.9973
	8	4	300.5	569.6	86.5	6.6	0.86	48.22	0.9972	47.84	0.9971
	12	8	266.3	569.6	76.6	7.4	0.88	48.02	0.9972	47.64	0.9971
<i>Tango_Duo</i>	4	0	485.1	49.4	11.5	4.3	0.80	51.51	0.9985	47.77	0.9965
	8	4	438.1	49.4	10.4	4.8	0.81	51.39	0.9985	47.66	0.9964
	12	8	395.4	49.4	9.4	5.3	0.83	51.31	0.9984	47.58	0.9964
<i>Tango_Solo_Female</i>	4	0	465.4	46.6	10.4	4.5	0.80	55.24	0.9993	49.04	0.9972
	8	4	416.4	46.6	9.3	5.0	0.82	55.07	0.9993	48.87	0.9971
	12	8	376.9	46.6	8.4	5.5	0.83	54.99	0.9993	48.79	0.9971
<i>Tango_Solo_Male</i>	4	0	383.4	45.5	8.4	5.4	0.83	57.93	0.9995	51.80	0.9980
	8	4	332.1	45.5	7.3	6.2	0.85	57.72	0.9995	51.59	0.9979
	12	8	299.7	45.5	6.6	6.9	0.86	57.58	0.9995	51.45	0.9979
<i>Tennis_Player</i>	4	0	452.5	47.6	10.4	4.6	0.81	55.78	0.9996	46.21	0.9959
	8	4	404.4	47.6	9.3	5.1	0.82	55.74	0.9995	46.17	0.9958
	12	8	364.6	47.6	8.4	5.7	0.84	55.70	0.9995	46.13	0.9958

The objective quality metrics support these qualitative observations. Table 1 shows the summarized quantitative metrics of the proposed framework. The method maintained an average Viewpoint PSNR of 53.00 dB across all test sequences, which indicates a near-lossless visual quality compared to the reference rendering. Specifically, the Tango sequences achieved PSNR values exceeding 56 dB, suggesting that our non-linear restoration logic (Log-domain for scale, Sigmoid for opacity) effectively mitigates quantization errors in sensitive attributes.

5 CONCLUSION

In this paper, we presented a standard-compliant streaming framework for 3D Gaussian Splatting (3DGS) content by integrating MPEG G-PCC compression with the glTF 2.0 container format. To overcome the limitation of G-PCC's integer-only arithmetic, this paper introduced a quantization-aware encoding scheme that adapts high-precision floating-point Gaussian attributes to the G-PCC pipeline. Furthermore, we defined a custom glTF extension (MPEG_GSC_EXT) to encapsulate the compressed bitstream and metadata, ensuring interoperability within the web 3D ecosystem.

Experimental results demonstrated that the proposed method achieves an average compression ratio of approximately 11.2:1 compared to raw PLY files, significantly alleviating the bandwidth bottleneck for immersive media transmission. Moreover, the proposed client-side rendering pipeline with on-the-fly de-quantization proved to be effective, maintaining an average Viewpoint PSNR of 27.8 dB and high structural similarity.

In future work, we plan to extend this format to support dynamic 4D Gaussian Splatting sequences, effectively addressing the tempo-

ral redundancy in moving scenes. Furthermore, we aim to enhance the packaging framework to accommodate multiple distinct Gaussian Splatting objects within a single container, enabling the composition of complex, multi-entity environments for fully immersive XR experiences.

ACKNOWLEDGMENTS

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. RS-2024-00457605).

This research was also supported by Culture, Sports and Tourism R&D Program through the Korea Creative Content Agency grant funded by the Ministry of Culture, Sports and Tourism in 2023 (Project Name: Development of content streaming copyright core technology on metaverse platform, Project Number: RS-2023-00223812, Contribution Rate: 50%)

REFERENCES

- [1] I. Bouazizi and T. Stockhammer. glTF as a Gaussian Splat format. Technical Report m73557, ISO/IEC JTC 1/SC 29/WG 3, June 2025. 2
- [2] J. M. Boyce et al. MPEG immersive video coding standard. *Proceedings of the IEEE*, 109(9):1521–1536, Sept. 2021. doi: 10.1109/JPROC.2021.3062590 1
- [3] ISO/IEC. Information technology — coded representation of immersive media — part 9: Geometry-based point cloud compression. Standard ISO/IEC 23090-9:202X, International Organization for Standardization and International Electrotechnical Commission, Geneva, Switzerland, 2021. ISO/IEC JTC 1/SC 29/WG 7. 1

- [4] J.-B. Jeong, S. Lee, and E.-S. Ryu. Delta QP allocation for MPEG immersive video. In *Proceedings of the IEEE International Conference on Big Data and Smart Computing (BigComp)*, 2022. [2](#)
- [5] J.-B. Jeong, S. Lee, and E.-S. Ryu. Implementing partial atlas selector for viewport-dependent MPEG immersive video streaming. In *Proceedings of the ACM Workshop on Network and Operating Systems Support for Digital Audio and Video (NOSSDAV)*, 2023. [1](#)
- [6] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), July 2023. doi: 10.1145/3592433 [1](#)
- [7] S. Lee, J.-B. Jeong, and E.-S. Ryu. Efficient group-based packing strategy for 6DoF immersive video streaming. In *Proceedings of the International Conference on Information Networking (ICOIN)*, 2022. [1](#)
- [8] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020. [1](#)
- [9] F. Robinet et al. gltf: Designing an open-standard runtime asset format. In *GPU Pro 360 Guide to 3D Engine Design*, pp. 243–260. AK Peters/CRC Press, 2018. [1](#)